

untangle::actuator

Generated by Doxygen 1.18.0

1 actuator	1
2 Topic Index	3
2.1 Topics	3
3 Namespace Index	5
3.1 Namespace List	5
4 Class Index	7
4.1 Class List	7
5 File Index	9
5.1 File List	9
6 Topic Documentation	11
6.1 namespace untangle: functions	11
6.1.1 Detailed Description	11
6.1.2 Function Documentation	11
6.1.2.1 connect()	11
6.1.2.2 bind() [1/2]	12
6.1.2.3 bind() [2/2]	13
7 Namespace Documentation	15
7.1 untangle Namespace Reference	15
7.1.1 Detailed Description	15
8 Class Documentation	17
8.1 untangle::actuator< action_t > Struct Template Reference	17
8.1.1 Detailed Description	18
8.1.2 Member Typedef Documentation	18
8.1.2.1 actions_t	18
8.1.2.2 results_t	18
8.1.3 Member Function Documentation	19
8.1.3.1 operator=()	19
8.1.3.2 type()	19
8.1.3.3 reset()	19
8.1.3.4 operator()()	19
8.1.3.5 invoke_action()	20
8.1.3.6 add() [1/2]	20
8.1.3.7 add() [2/2]	20
8.1.3.8 remove() [1/2]	21
8.1.3.9 remove() [2/2]	21
8.1.3.10 is_connected()	22
8.1.3.11 has_action()	22
8.2 untangle::invalid_action Struct Reference	22
8.2.1 Detailed Description	23

8.2.2 Constructor & Destructor Documentation	23
8.2.2.1 invalid_action()	23
8.2.3 Member Function Documentation	23
8.2.3.1 what()	23
9 File Documentation	25
9.1 actuator.hpp	25
Index	29

Chapter 1

actuator

An actuator is a generic callable that may invoke a list of actions(callables of type `std::function<...>`).

Its goal is to provide a light and simple mechanism to connect components that want to communicate with each other. It works similar as the "signal and slots" mechanisms from other frameworks, but much simplified.

Modern programming languages do not offer class intrinsic interfaces that would improve significantly how objects are connected and how they communicate. Instead, languages, like C++, are using external interfaces, that are not only cluttering the code, but they don't feel "natural" as they are opposed to what an interface is in reality. Basically the objects and interfaces are not separated, and the interface is intrinsic to the object(think on the cogs in a mechanism). A generic callable is an attempt to mitigate this problem. `async` implementation is such use case.

An actuator provides the results of all actions in a vector, in the same order as the actions were added.

Example: how to use actuator instead of polymorphism

```
void rotate_shapes(const std::vector<shape*>& shapes, int angle)
{
    for (const auto& s : shapes)
    {
        s->rotate(angle);
    }
}

std::shared_ptr<triangle> t(new triangle);
std::shared_ptr<circle> c(new circle);
std::shared_ptr<square> s(new square);

// using polymorphism
std::vector<shape*> shapes;
shapes.push_back(t.get());
shapes.push_back(c.get());
shapes.push_back(s.get());

rotate_shapes(shapes, 10);

// using actuator
auto action1 = untangle::bind(t, &triangle::rotate);
auto action2 = untangle::bind(c, &circle::rotate);
auto action3 = untangle::bind(s, &square::rotate);
auto actuator_rotate = untangle::connect(action1, action2, action3);
actuator_rotate(20);
```

For convenience there are provided helpers methods to "connect" to an initial list of "actions", or to create bindings to class methods.

Please check the manual in [doc/refman.pdf](#) for further references.

Chapter 2

Topic Index

2.1 Topics

Here is a list of all topics with brief descriptions:

namespace untangle: functions [11](#)

Chapter 3

Namespace Index

3.1 Namespace List

Here is a list of all documented namespaces with brief descriptions:

untangle	Interface to untangle::actuator functor	15
--------------------------	---	--------------------

Chapter 4

Class Index

4.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

untangle::actuator< action_t >	
An actuator is a functor that can trigger a dynamic list of actions (of type std::↵ function<...>)	17
untangle::invalid_action	
Invalid action exception	22

Chapter 5

File Index

5.1 File List

Here is a list of all documented files with brief descriptions:

actuator.hpp	25
--	----

Chapter 6

Topic Documentation

6.1 namespace untangle: functions

Functions

- `template<typename action_t, typename ... Actions>`
`actuator< action_t > untangle::connect (action_t &A1, Actions &... An)`
Creates an actuator holding an initial list of actions.
- `template<typename class_t, typename T, typename action_t = std::function<typename function_remove_const<↵`
`T>::type>>`
`action_t untangle::bind (const std::shared_ptr< class_t > &obj, T class_t::*method)`
Binding to a class function member.
- `template<typename class_t, typename T, typename action_t = std::function<typename function_remove_const<↵`
`T>::type>>`
`action_t untangle::bind (class_t *obj, T class_t::*method)`
Binding to a class method.

6.1.1 Detailed Description

6.1.2 Function Documentation

6.1.2.1 connect()

```
template<typename action_t, typename ... Actions>
actuator< action_t > untangle::connect (
    action_t & A1,
    Actions &... An)
```

Creates an actuator holding an initial list of actions.

Parameters

A1	- The first action. It is specified as <code>std::function<...></code> .
----	--

An	- Any number of further actions, of the same type as A1.
----	--

Returns

An [actuator](#).

Example:

```
void rotate_shapes(const std::vector<shape*>& shapes, int angle)
{
    for (const auto& s : shapes)
    {
        s->rotate(angle);
    }
}

auto action1 = untangle::bind(t, &triangle_mock::rotate);
auto action2 = untangle::bind(c, &circle_mock::rotate);
auto action3 = untangle::bind(s, &square_mock::rotate);

auto actuator_rotate = untangle::connect(action1, action2, action3);
actuator_rotate(20);
```

6.1.2.2 bind() [1/2]

```
template<typename class_t, typename T, typename action_t = std::function<typename function_remove_const<T>::<
type>>>
action_t untangle::bind (
    const std::shared_ptr< class_t > & obj,
    T class_t::* method)
```

Binding to a class function member.

It returns a `std::function(lambda)` that wraps the function member. It may be used to provide an action for [connect\(\)](#) or [actuator::add\(\)](#).

Remarks

It requires a shared pointer to the class type. A `std::weak_ptr` to it is captured internally in a lambda, so the binding can always check whether the shared object is still alive without keeping it alive itself. Therefore, it is safe to use actions provided by this binding inside an [actuator](#), and safe for the action to outlive the caller's shared pointer.

Parameters

obj	- Class object.
method	- Pointer to function member. It is specified as <code>&<class type>::<function member></code>

Returns

`action_t` - A `std::function` that wraps the pointer to function member.

Remarks

If the class object gets invalid, invoking this binding will throw an exception of type [invalid_action](#).

6.1.2.3 bind() [2/2]

```
template<typename class_t, typename T, typename action_t = std::function<typename function_remove_const<T>::↵
type>>
action_t untangle::bind (
    class_t * obj,
    T class_t::* method)
```

Binding to a class method.

Attention

It is not safe to use this binding to provide actions to an [actuator](#) . The class object is provided through a pointer type. This pointer is captured internally in a lambda, so it can not be checked if it gets null.

Remarks

It is provided for convenience of use: within a class it is safe to create bindings through this pointer.

Parameters

obj	- Pointer to class.
method	- Pointer to function member. It is specified as &<class type>::<function member>

Returns

action_t - A std::function that wraps the pointer to function member.

Chapter 7

Namespace Documentation

7.1 untangle Namespace Reference

Interface to [untangle::actuator](#) functor.

Classes

- struct [invalid_action](#)
Invalid action exception.
- struct [actuator](#)
An actuator is a functor that can trigger a dynamic list of actions (of type `std::function<...>`).

Functions

- `template<typename action_t, typename ... Actions>`
[actuator](#)< action_t > [connect](#) (action_t &A1, Actions &... An)
Creates an actuator holding an initial list of actions.
- `template<typename class_t, typename T, typename action_t = std::function<typename function_remove_const<↵
T>::type>>`
`action_t` [bind](#) (const std::shared_ptr< class_t > &obj, T class_t::*method)
Binding to a class function member.
- `template<typename class_t, typename T, typename action_t = std::function<typename function_remove_const<↵
T>::type>>`
`action_t` [bind](#) (class_t *obj, T class_t::*method)
Binding to a class method.

7.1.1 Detailed Description

Interface to [untangle::actuator](#) functor.

Chapter 8

Class Documentation

8.1 untangle::actuator< action_t > Struct Template Reference

An actuator is a functor that can trigger a dynamic list of actions (of type `std::function<...>`).

```
#include <actuator.hpp>
```

Public Types

- using `actions_t` = `std::list<action_t*>`
Actions container type.
- using `results_t` = `std::vector<typename result_t::type>`
Results container type.

Public Member Functions

- `actuator & operator= (const actuator &other)=default`
Assignment operator.
- `action_t type () const`
Helper method to access the action type of this object.
- `void reset ()`
Remove all actions and any stored results.
- `template<typename ... Args>`
`void operator() (Args &&... args)`
The call operator.
- `template<typename ... Args>`
`void invoke_action (const std::string &name, Args &&... args)`
Invokes one single action associated with a key.
- `void add (action_t *action)`
Add an action to the actions list.
- `void add (const std::string &name, action_t *action)`
Add action to the actions map associated with a name.
- `void remove (const action_t *action)`
Remove an action from the actions list.
- `void remove (const std::string &name)`
Remove an action from actions map.
- `bool is_connected () const`
Check if this actuator is "connected" with other actions.
- `bool has_action (const std::string &name) const`
Check if there is certain named action.

Public Attributes

- [actions_t](#) actions
Actions list.
- [actions_map_t](#) [actions_map](#)
Named actions map.
- [results_t](#) results
Actions return values list.

8.1.1 Detailed Description

```
template<typename action_t>
struct untangle::actuator< action_t >
```

An actuator is a functor that can trigger a dynamic list of actions (of type `std::function<...>`).

Remarks

An actuator object can be constructed with an initial list of actions by [connect\(\)](#).

Template Parameters

<code>action_t</code>	Action type. It is specified as <code>std::function<...></code> .
-----------------------	---

8.1.2 Member Typedef Documentation

8.1.2.1 `actions_t`

```
template<typename action_t>
using untangle::actuator< action_t >::actions_t = std::list<action_t*>
```

Actions container type.

Remarks

The elements stored are of pointer type, that is required to implement the [remove\(\)](#) operation. `std::function` supports only equality operator for nullptr (two `std::function(s)` can not compare).

8.1.2.2 `results_t`

```
template<typename action_t>
using untangle::actuator< action_t >::results_t = std::vector<typename result_t::type>
```

Results container type.

It holds the return values of the actions that have a non-void return type. Upon the actuator invocation, the returns can be extracted from [results](#).

8.1.3 Member Function Documentation

8.1.3.1 operator=()

```
template<typename action_t>
actuator & untangle::actuator< action_t >::operator= (
    const actuator< action_t > & other) [default]
```

Assignment operator.

Example:

```
const auto t = std::make_shared<triangle_mock>();
const auto c = std::make_shared<circle_mock>();
const auto s = std::make_shared<square_mock>();

EXPECT_CALL(*t, rotate(testing::_)).WillOnce(testing::Return());
EXPECT_CALL(*c, rotate(testing::_)).WillOnce(testing::Return());
EXPECT_CALL(*s, rotate(testing::_)).WillOnce(testing::Return());
auto action1 = untangle::bind(t, &triangle_mock::rotate);
auto action2 = untangle::bind(c, &circle_mock::rotate);
auto action3 = untangle::bind(s, &square_mock::rotate);

auto actuator_rotate = untangle::connect(action1, action2, action3);
untangle::actuator<decltype(actuator_rotate.type())> actuator_rotate_1 = actuator_rotate;
actuator_rotate_1(20);

testing::Mock::VerifyAndClearExpectations(t.get());
testing::Mock::VerifyAndClearExpectations(c.get());
testing::Mock::VerifyAndClearExpectations(s.get());
```

8.1.3.2 type()

```
template<typename action_t>
action_t untangle::actuator< action_t >::type () const [inline]
```

Helper method to access the action type of this object.

Note

Intended to be used in expressions by `decltype(<actuator instance>.type())`

Returns

action_t

8.1.3.3 reset()

```
template<typename action_t>
void untangle::actuator< action_t >::reset () [inline]
```

Remove all actions and any stored results.

After this call the actuator is empty: `actuator::is_connected()` returns false.

8.1.3.4 operator()()

```
template<typename action_t>
template<typename ... Args>
void untangle::actuator< action_t >::operator() (
    Args &&... args) [inline]
```

Parameters

args	- Arguments list must match the action arity.
------	---

8.1.3.5 invoke_action()

```
template<typename action_t>
template<typename ... Args>
void untangle::actuator< action_t >::invoke_action (
    const std::string & name,
    Args &&... args) [inline]
```

Invokes one single action associated with a key.

Parameters

name	- Key associated with the action.
args	- Arguments list must match the action arity.

8.1.3.6 add() [1/2]

```
template<typename action_t>
void untangle::actuator< action_t >::add (
    action_t * action) [inline]
```

Add an action to the actions list.

Parameters

action	- Action to be added.
--------	-----------------------

Example:

```
const auto t = std::make_shared<triangle_mock>();
const auto c = std::make_shared<circle_mock>();
const auto s = std::make_shared<square_mock>();

EXPECT_CALL(*t, rotate(testing::_)).Times(2).WillRepeatedly(testing::Return());
EXPECT_CALL(*c, rotate(testing::_)).WillOnce(testing::Return());
EXPECT_CALL(*s, rotate(testing::_)).WillOnce(testing::Return());
auto action1 = untangle::bind(t, &triangle_mock::rotate);
auto action2 = untangle::bind(c, &circle_mock::rotate);
auto action3 = untangle::bind(s, &square_mock::rotate);

auto actuator_rotate = untangle::connect(action1, action2, action3);
actuator_rotate.add(&action1);
actuator_rotate(20);

testing::Mock::VerifyAndClearExpectations(t.get());
testing::Mock::VerifyAndClearExpectations(c.get());
testing::Mock::VerifyAndClearExpectations(s.get());
```

8.1.3.7 add() [2/2]

```
template<typename action_t>
void untangle::actuator< action_t >::add (
```


Parameters

name	- Name of the action.
action	- Action to be added.

8.1.3.8 remove() [1/2]

```
template<typename action_t>
void untangle::actuator< action_t >::remove (
    const action_t * action) [inline]
```

Remove an action from the actions list.

An invalid action (empty std::function) is implicitly removed when `operator()()` is invoked.

Parameters

action	- Action to be removed.
--------	-------------------------

Example:

```
const auto t = std::make_shared<triangle_mock>();
const auto c = std::make_shared<circle_mock>();
const auto s = std::make_shared<square_mock>();

EXPECT_CALL(*t, rotate(testing::_)).Times(0);
EXPECT_CALL(*c, rotate(testing::_)).WillOnce(testing::Return());
EXPECT_CALL(*s, rotate(testing::_)).WillOnce(testing::Return());
auto action1 = untangle::bind(t, &triangle_mock::rotate);
auto action2 = untangle::bind(c, &circle_mock::rotate);
auto action3 = untangle::bind(s, &square_mock::rotate);

auto actuator_rotate = untangle::connect(action1, action2, action3);
actuator_rotate.remove(&action1);
actuator_rotate(50);

testing::Mock::VerifyAndClearExpectations(t.get());
testing::Mock::VerifyAndClearExpectations(c.get());
testing::Mock::VerifyAndClearExpectations(s.get());
```

8.1.3.9 remove() [2/2]

```
template<typename action_t>
void untangle::actuator< action_t >::remove (
    const std::string & name) [inline]
```

Remove an action from actions map.

Parameters

name	- Name of the action to remove.
------	---------------------------------

8.1.3.10 is_connected()

```
template<typename action_t>
bool untangle::actuator< action_t >::is_connected () const [inline]
```

Check if this actuator is "connected" with other actions.

Returns

- true - if the `actuator::actions` list is not empty.
- false - if the `actuator::actions` list is empty.

8.1.3.11 has_action()

```
template<typename action_t>
bool untangle::actuator< action_t >::has_action (
    const std::string & name) const [inline]
```

Check if there is certain named action.

Parameters

name	- Name associated with the action.
------	------------------------------------

Returns

- true - if name can be found in `actuator::actions_map`
- false - if name can not be found in `actuator::actions_map`

The documentation for this struct was generated from the following file:

- `actuator.hpp`

8.2 untangle::invalid_action Struct Reference

Invalid action exception.

```
#include <actuator.hpp>
```

Public Member Functions

- `invalid_action` (std::string text)
Construct a new invalid action object.
- `const char * what () const` noexcept override
The message text describing the reason of this exception.

8.2.1 Detailed Description

Invalid action exception.

Remarks

An action may be provided as a binding to a class function member, by using [untangle::bind\(\)](#). When the class object gets invalid, invoking the action will raise an exception to this type.

8.2.2 Constructor & Destructor Documentation

8.2.2.1 invalid_action()

```
untangle::invalid_action::invalid_action (  
    std::string text) [inline], [explicit]
```

Construct a new invalid action object.

Parameters

text	- A message text, describing the reason of this exception.
------	--

8.2.3 Member Function Documentation

8.2.3.1 what()

```
const char * untangle::invalid_action::what () const [inline], [override], [noexcept]
```

The message text describing the reason of this exception.

Returns

const char* - The message text.

The documentation for this struct was generated from the following file:

- actuator.hpp

Chapter 9

File Documentation

9.1 actuator.hpp

```
00001 // Copyright (c) 2018 Nicolae Popescu. MIT License.
00002
00006 #pragma once
00007
00008 #include <vector>
00009 #include <list>
00010 #include <map>
00011 #include <string>
00012 #include <utility>
00013 #include <functional>
00014 #include <memory>
00015 #include <iostream>
00016 #include <type_traits>
00017 #include <cassert>
00018 #include <exception>
00019
00020 namespace untangle
00021 {
00022     // exception
00030 struct invalid_action : std::exception
00031 {
00037     explicit invalid_action(std::string text) : message(std::move(text)){}
00038
00044     const char* what() const noexcept override { return message.c_str(); }
00045
00046 private:
00047     std::string message;
00048 };
00049
00057 template<typename action_t>
00058 struct actuator final
00059 {
00066     using actions_t = std::list<action_t*>;
00067     using actions_map_t = std::map<std::string, action_t*>;
00068     using result_t = std::conditional<std::is_void<typename action_t::result_type>::value, int, typename
    action_t::result_type>;
00075     using results_t = std::vector<typename result_t::type>;
00076
00077     actions_t actions;
00078     actions_map_t actions_map;
00079     results_t results;
00080
00081     actuator() = default;
00082     actuator(const actuator&) = default;
00083     actuator(actuator&&) noexcept = default;
00084     ~actuator() = default;
00091     actuator& operator=(const actuator& other) = default;
00092     actuator& operator=(actuator&&) noexcept = default;
00093
00101     action_t type() const
00102     {
00103         return nullptr;
00104     }
00105
00111     void reset()
00112     {
00113         actions.clear();
```

```

00114     actions_map.clear();
00115     results.clear();
00116 }
00117
00125 template<typename ...Args>
00126 void operator()(Args&&... args)
00127 {
00128     results.clear();
00129
00130     // Dead bindings are collected here and dropped after the loop.
00131     // The actuator does not own the actions it points at, so it must never
00132     // write through action_t* into a std::function belonging to the caller.
00133     std::vector<action_t*> dead_actions;
00134
00135     for (const auto& action : actions)
00136     {
00137         // A null pointer or an empty std::function can never be invoked. Calling an
00138         // empty one throws std::bad_function_call, which is not an invalid_action and
00139         // would escape this operator, so drop it instead of invoking it.
00140         if (action == nullptr || !*action)
00141         {
00142             dead_actions.push_back(action);
00143             continue;
00144         }
00145
00146         try
00147         {
00148             if constexpr (std::is_same_v<typename action_t::result_type, void>) {
00149                 (*action)(std::forward<Args>(args)...);
00150             } else {
00151                 results.push_back((*action)(std::forward<Args>(args)...));
00152             }
00153         }
00154         catch (const invalid_action& ia)
00155         {
00156             std::cout << ia.what() << std::endl;
00157             dead_actions.push_back(action);
00158         }
00159     }
00160
00161     for (const auto& dead_action : dead_actions)
00162     {
00163         actions.remove(dead_action);
00164     }
00165 }
00166
00173 template<typename ...Args>
00174 void invoke_action(const std::string& name, Args&&... args)
00175 {
00176     results.clear();
00177     const auto& it = actions_map.find(name);
00178     if (it != actions_map.end())
00179     {
00180         try
00181         {
00182             if constexpr (std::is_same_v<typename action_t::result_type, void>) {
00183                 (*it->second)(std::forward<Args>(args)...);
00184             } else {
00185                 results.push_back((*it->second)(std::forward<Args>(args)...));
00186             }
00187         }
00188         catch (const invalid_action& ia)
00189         {
00190             std::cout << ia.what() << std::endl;
00191             actions_map.erase(name);
00192         }
00193     }
00194 }
00195
00204 void add(action_t* action)
00205 {
00206     actions.push_back(action);
00207 }
00208
00215 void add(const std::string& name, action_t* action)
00216 {
00217     actions_map.emplace(name, action);
00218 }
00219
00230 void remove(const action_t* action)
00231 {
00232     actions.remove_if([&action](const auto& a)
00233     {
00234         return (action == a);
00235     });
00236 }
00237

```

```

00243 void remove(const std::string& name)
00244 {
00245     actions_map.erase(name);
00246 }
00247
00254 bool is_connected() const { return !actions.empty() || !actions_map.empty(); }
00255
00263 bool has_action(const std::string& name) const { return actions_map.find(name) != actions_map.end(); }
00264 };
00265
00280 template<typename action_t, typename ...Actions>
00281 actuator<action_t> connect(action_t& A1, Actions&... An)
00282 {
00283     using actuator_t = untangle::actuator<action_t>;
00284     actuator_t actuator;
00285     actuator.actions = {&A1, &An...};
00286
00287     // remove empty actions
00288     actuator.actions.remove_if([](const auto& action)
00289     {
00290         return (*action == nullptr);
00291     });
00292     return actuator;
00293 }
00294
00295 template <typename actuator_t>
00296 void remove_empty_actions(actuator_t& actuator)
00297 {
00298     std::vector<typename actuator_t::actions_map_t::const_iterator> removable_iterators;
00299     for (auto it = actuator.actions_map.begin(); it != actuator.actions_map.end(); ++it)
00300     {
00301         if (it->second == nullptr)
00302         {
00303             removable_iterators.push_back(it);
00304         }
00305     }
00306     for (auto it: removable_iterators)
00307     {
00308         actuator.actions_map.erase(it);
00309     }
00310 }
00311
00312 template<typename key_t, typename action_t, typename ...Actions>
00313 actuator<action_t> connect(std::pair<key_t, action_t*> A1, Actions... An)
00314 {
00315     using actuator_t = untangle::actuator<action_t>;
00316     actuator_t actuator;
00317     actuator.actions_map = {A1, An...};
00318
00319     // remove empty actions
00320     remove_empty_actions(actuator);
00321     return actuator;
00322 }
00323
00324 // generic helpers to remove const qualifier from a function type,
00325 // for instance const member functions
00326 template <typename T>
00327 struct function_remove_const;
00328
00329 template <typename R, typename... Args>
00330 struct function_remove_const<R(Args...)>
00331 {
00332     using type = R(Args...);
00333 };
00334
00335 template <typename R, typename... Args>
00336 struct function_remove_const<R(Args...)const>
00337 {
00338     using type = R(Args...);
00339 };
00340
00344
00363 template <typename class_t, typename T, typename action_t = std::function<typename
    function_remove_const<T>::type>
00364 action_t bind(const std::shared_ptr<class_t>& obj, T class_t::* method)
00365 {
00366     return [wp = std::weak_ptr<class_t>(obj), method](auto&&... args) -> typename action_t::result_type
00367     {
00368         // lock() also keeps the object alive for the duration of the call
00369         if (const auto obj_ = wp.lock())
00370         {
00371             return ((*obj_).*method)(std::forward<decltype(args)>(args)...);
00372         }
00373         else
00374         {
00375             //inform the actuator about dead binding
00376             throw invalid_action("bind::method: invalid object");

```

```
00377     }
00378   };
00379 }
00380
00394 template <typename class_t, typename T, typename action_t = std::function<typename
      function_remove_const<T>::type»
00395 action_t bind(class_t* obj, T class_t::* method)
00396 {
00397     assert(obj != nullptr);
00398     return [obj, method](auto&&... args) -> typename action_t::result_type
00399     {
00400         return ((obj)->*method)(std::forward<decltype(args)>(args)...);
00401     };
00402 }
00403
00404 }
```


Index

- actions__t
 - untangle::actuator< action__t >, [18](#)
- actuator, [1](#)
- add
 - untangle::actuator< action__t >, [20](#)
- bind
 - namespace untangle: functions, [12](#)
- connect
 - namespace untangle: functions, [11](#)
- has__action
 - untangle::actuator< action__t >, [22](#)
- invalid__action
 - untangle::invalid__action, [23](#)
- invoke__action
 - untangle::actuator< action__t >, [20](#)
- is__connected
 - untangle::actuator< action__t >, [21](#)
- namespace untangle: functions, [11](#)
 - bind, [12](#)
 - connect, [11](#)
- operator()
 - untangle::actuator< action__t >, [19](#)
- operator=
 - untangle::actuator< action__t >, [19](#)
- remove
 - untangle::actuator< action__t >, [21](#)
- reset
- results__t
 - untangle::actuator< action__t >, [18](#)
- type
 - untangle::actuator< action__t >, [19](#)
- untangle, [15](#)
- untangle::actuator< action__t >, [17](#)
 - actions__t, [18](#)
 - add, [20](#)
 - has__action, [22](#)
 - invoke__action, [20](#)
 - is__connected, [21](#)
 - operator(), [19](#)
 - operator=, [19](#)
 - remove, [21](#)
 - reset, [19](#)
 - results__t, [18](#)
 - type, [19](#)
 - untangle::invalid__action, [22](#)
 - invalid__action, [23](#)
 - what, [23](#)
- what
 - untangle::invalid__action, [23](#)